

OPTIMA

Intelligent Test-Case Optimization & Coverage Engine

PROFESSIONAL PROJECT REPORT

Project	OPTIMA — Intelligent Test-Case Optimization & Coverage Engine
Problem	Problem 97 — Coding Contest Test-Case Optimizer
Algorithms	Sorting • Greedy • 0/1 Knapsack
Technology	React • Python Flask • SQLite
Domain	Software Testing & Algorithmic Optimization
Team	_____
Institution	_____

Prepared for Hackathon / Academic Project Evaluation

1. ABSTRACT & INTRODUCTION

1.1 Abstract

OPTIMA is an algorithm-centric software test-case optimization system designed for situations where a large test suite must be executed under a limited execution-time budget. The system accepts test-case attributes such as topic, edge-case status, execution time and priority, stores them persistently, and applies Sorting, Greedy and 0/1 Knapsack techniques to identify valuable test-case combinations. It then compares the algorithmic results and measures topic and edge-case coverage.

The project emphasizes transparency rather than returning only a final number. Important decisions, selected and rejected cases, optimization metrics, coverage indicators and performance information are presented through an interactive interface. The project is based on the DAA Hackathon Problem 97: Coding Contest Test-Case Optimizer.

1.2 Introduction

Software testing is essential for maintaining application quality, but complete test suites can become large and expensive to execute. When release schedules or computational resources impose a strict time limit, a systematic selection strategy is required. OPTIMA models this as a constrained optimization problem: select a compact set of test cases that provides high value and broad coverage without exceeding the available execution budget.

1.3 Project Vision

The central vision of OPTIMA is to make test-suite selection measurable, algorithmic and understandable.

2. PROBLEM STATEMENT, OBJECTIVES & SCOPE

2.1 Problem Statement

Given a collection of software test cases with different execution costs, priorities, topics and edge-case characteristics, select a compact subset that maximizes useful value and coverage while remaining within a limited execution-time budget.

2.2 Objectives

- Represent test cases using appropriate data attributes and persistent storage.
- Organize test cases using sorting for ranking and selection.
- Apply a Greedy strategy to make locally best feasible selections.
- Apply 0/1 Knapsack to optimize value under the execution budget.
- Compare the behavior and results of the selected algorithms.
- Measure topic and edge-case coverage.
- Expose decision traces, metrics and performance information.
- Present the final result clearly through the application interface and reports.

2.3 Scope

- Test-case creation, storage and deletion.
- Priority, execution-time and efficiency based sorting.
- Budget-constrained optimization.
- Greedy and 0/1 Knapsack result generation.
- Coverage and analytics visualization.

- PDF and Excel-compatible reporting.

The current project focuses on algorithmic optimization and demonstration; advanced machine-learning-based prioritization and CI/CD integration are future-scope items.

3. FEATURES & FUNCTIONAL REQUIREMENTS

3.1 Major Features

Feature	Description
Test Case Repository	Store and manage test-case records.
Sorting & Ranking	Sort by priority, execution time and efficiency.
Greedy Optimization	Select high-value feasible cases using a local decision rule.
0/1 Knapsack	Optimize total value without exceeding the execution budget.
Algorithm Comparison	Compare Greedy and Knapsack outputs.
Coverage Analysis	Measure topic and edge-case coverage.
Decision Trace	Show selected/rejected decisions and reasoning.
Analytics	Display counts, distributions, coverage and performance metrics.
Report Export	Generate a printable PDF report and Excel-compatible output.

3.2 Functional Requirements

- The system shall accept test-case name, topic, edge-case status, execution time and priority.
- The system shall persist test cases in SQLite.
- The system shall support adding and deleting test cases.
- The system shall allow sorting using supported criteria.
- The system shall accept an execution budget.
- The system shall execute Greedy and 0/1 Knapsack optimization.
- The system shall display selected cases and optimization metrics.
- The system shall calculate topic and edge-case coverage.
- The system shall expose algorithm decisions and performance information.
- The system shall provide report/export functionality.

4. ALGORITHMS & METHODOLOGY

4.1 Input Model

Attribute	Meaning	Example
Name	Test-case identifier	Payment Failure
Topic	Functional area	Payment
Edge Case	Edge-condition indicator	Yes
Execution Time	Resource cost	3 sec
Priority	Relative value	9
Budget	Maximum execution resource	6 sec

4.2 Overall Methodology

Input → Store → Sort → Optimize → Compare → Measure Coverage → Analyze → Report

4.3 Sorting

Sorting organizes test cases according to priority, execution time or an efficiency measure. This makes ranking and the subsequent greedy selection process systematic. The implementation also exposes sorting-related comparisons/timing where appropriate.

4.4 Greedy Optimization

The Greedy strategy makes the best locally feasible choice according to a defined selection rule. In OPTIMA, candidate cases are considered using value relative to execution cost and are accepted when they fit the remaining budget. The system records selections and rejections so the process can be inspected.

4.5 0/1 Knapsack Optimization

The Knapsack formulation treats every test case as a binary decision: selected or not selected. The objective is to maximize total value without exceeding the execution-time budget. The dynamic-programming formulation evaluates budget states and reconstructs the selected set.

4.6 Greedy vs Knapsack

Aspect	Greedy	0/1 Knapsack
Strategy	Local best feasible choice	Combination-based optimization
Typical computation	Lower	Higher
Budget	Respected	Respected
Result	Feasible high-value set	Maximum-value feasible set for the formulation
Trace	Selection/rejection steps	DP optimization/reconstruction

4.7 Coverage Method

$$\text{Topic Coverage} = \text{Covered Topics} / \text{Total Topics} \times 100$$

$$\text{Edge-Case Coverage} = \text{Selected Edge Cases} / \text{Total Edge Cases} \times 100$$

These measures provide a direct view of how broadly the selected suite exercises the available application areas and edge conditions.

5. TECHNOLOGIES & IMPLEMENTATION

5.1 Technology Stack

Layer	Technology	Purpose
Frontend	React + Vite	Interactive user interface
UI	Custom CSS + Framer Motion	Visual design and animations
Icons	Lucide	Interface iconography
Backend	Python Flask	REST API and application logic
Database	SQLite	Persistent test-case storage

5.2 System Architecture

Frontend: React	User interaction, test-case management, algorithms/results display, analytics
REST API: Flask	Receives requests and connects the interface with application/database logic
Database: SQLite	Persists test-case records

5.3 Implementation Flow

- React collects test-case data and the execution budget.
- Flask exposes REST endpoints for test-case persistence and retrieval.
- SQLite stores the test-case repository.
- The frontend runs the optimization and displays algorithmic decisions and results.
- Coverage and analytics are calculated from the available and selected test cases.
- Reports can be prepared from the resulting metrics.

6. RESULTS, TESTING & PERFORMANCE

6.1 Expected / Demonstration Result

For a demonstration, consider an execution budget of 10 minutes. An illustrative comparison can be presented as follows; actual values should be replaced with the final run's measured output.

Metric	Greedy	0/1 Knapsack
Selected Cases	4	5
Execution Time	9 min	10 min
Total Value	30	35
Coverage	75%	90%

6.2 Testing Strategy

Test Area	Validation
Backend Health	API root endpoint returns online status.
Create Test Case	New test case is inserted and returned with an ID.
Read Test Cases	Stored cases are loaded from the backend.
Persistence	Data remains available after refresh/reload.
Delete Test Case	Selected record is removed from the repository.
Sorting	Cases reorder according to the selected criterion.
Greedy	Selection respects the execution budget and produces a decision trace.
Knapsack	Selected set respects the budget and produces an optimization result.
Coverage	Topic and edge-case metrics are calculated from the selected set.

6.3 Performance & Complexity

Component	Complexity / Measure
Sorting	$O(n \log n)$
Greedy	$O(n \log n)$ when sorting is included
0/1 Knapsack	$O(n \times B)$
Coverage Analysis	$O(n)$
Performance View	Algorithm execution timing can be exposed for comparison

6.4 Practical Outcome

The project demonstrates a structured method for reducing unnecessary test execution while maintaining focus on valuable functionality and edge cases. It also makes algorithm decisions visible,

allowing users to understand how the selected test suite was produced.

7. CONCLUSION, FUTURE SCOPE & REFERENCES

7.1 Conclusion

OPTIMA converts software test-case selection into a DAA-based constrained optimization problem. Sorting supports ranking and organization, Greedy provides a fast locally feasible strategy, and 0/1 Knapsack provides a budget-constrained optimization approach. The system then compares results, evaluates coverage and exposes decision traces through an interactive interface.

The project therefore demonstrates not only the implementation of DAA algorithms but also their application to a practical software-testing problem.

7.2 Future Scope

- Historical failure-rate based prioritization.
- Automated test-case generation.
- Risk-based test selection.
- Integration with CI/CD pipelines.
- Jenkins / GitHub Actions integration.
- Large-scale test-suite optimization.
- Historical comparison of optimization runs.

7.3 References

- DAA Hackathon source document — Problem 97: Coding Contest Test-Case Optimizer.
- Project documentation — OPTIMA: Intelligent Test-Case Optimization & Coverage Engine.
- Project implementation — React frontend, Python Flask backend and SQLite database.

“Test smarter, not just more.”

— END OF REPORT —